URBAN BLACK

DYNAMIC PRICING ENGINE

Requirements Document

Rider App — Pricing & Revenue Domain (MVP)
Date: 24 March 2026

1. Purpose

The purpose of this document is to define all data, system, and integration requirements needed to build and deploy the Dynamic Pricing Engine for the Urban Black rider application.

The engine will compute in real time:

- Zone-level demand-supply surge multiplier
- Final fare estimate shown to the rider at booking
- Pricing adjustments based on time, weather, events, and driver availability

These outputs will support real-time fare calculation, booking conversion optimisation, and equitable driver incentivisation across all trip categories served by Urban Black.

2. Scope

In Scope

- Real-time demand-supply ratio computation per geospatial zone
- Surge multiplier calculation engine (base + contextual modifiers)
- Final fare estimation and breakdown for rider-facing booking screen
- Integration with Kafka event pipeline and booking backend
- Surge zone state publishing for consumption by other services (Long Trip Optimization Model, Driver App)
- Model inference service exposed as REST API
- Surge cap enforcement and regulatory override mechanism

Out of Scope

- Long-distance or intercity trip fare optimization (handled by Long Trip Optimization Model)
- Driver dispatch and fleet management logic
- Frontend UI/UX implementation for fare display
- Payment processing and settlement
- Historical fare auditing or dispute resolution workflows

3. Pricing Output Requirements

Target Outputs

Please confirm the following outputs are required for the MVP:

- Surge multiplier per zone
  - Mandatory — float, typically 1.0–4.0, configurable max cap
- Final fare estimate shown at booking
  - Mandatory — in local currency, inclusive of base fare + surge + applicable tolls
- Fare breakdown object
  - Mandatory — itemised base fare, surge component, toll component, platform fee
- Surge zone state event (published to Kafka)
  - Mandatory — consumed by Long Trip Optimization Model and Driver App
- Rider-facing surge indicator flag
  - Optional for MVP — boolean to trigger UI surge banner

Pricing Tiers & Multiplier Bands

Tier	Demand/Supply Ratio	Surge Multiplier	Rider Message
Normal	< 1.2	1.0×	Standard fare
Mild Surge	1.2 – 1.6	1.1× – 1.3×	High demand
Moderate Surge	1.6 – 2.2	1.3× – 1.8×	Very high demand
Peak Surge	2.2 – 3.0	1.8× – 2.5×	Peak surge active
Hard Cap	> 3.0	2.5× (max)	Capped surge

Computation Granularity

- Per hexagonal geospatial zone (H3 resolution 7 or equivalent grid)
- Recomputed every 60 seconds per zone or on-demand per ride request
- Multiplier applies to zone containing pickup coordinate

Computation Frequency

Please confirm the preferred trigger model:

- Event-driven: recomputed on each ride_request and driver_status Kafka event
- Scheduled batch: recomputed every 60 seconds for all active zones
- Hybrid (recommended): scheduled batch for zone state; per-request for final fare calculation

4. Input Data Requirements

The pricing engine requires the following input signals from backend services and Kafka streams.

A. Zone Demand & Supply Features (Mandatory)

#	Field	Description
1	zone_id	H3 hexagonal zone identifier for pickup location
2	active_ride_requests_in_z	Count of unfulfilled ride requests in zone (last 2 min)

	one	
3	available_drivers_in_zone	Count of online drivers within zone or adjacent zones
4	demand_supply_ratio	active_requests / max(available_drivers, 1)
5	avg_driver_eta_in_zone_min	Mean ETA of nearest available drivers in zone
6	zone_cancellation_rate_pct	Rider cancellation rate in zone (last 15 min)

B. Time Features (Mandatory)

#	Field	Description
1	timestamp	UTC ISO8601 timestamp of ride request
2	hour_of_day	Integer 0–23
3	day_of_week	Integer 0=Monday – 6=Sunday
4	is_peak_hour	Boolean — morning (7–10) or evening (17–20) window
5	is_holiday	Boolean — sourced from city holiday calendar
6	is_weekend	Boolean

C. Trip Features (Mandatory)

#	Field	Description
1	pickup_latitude	Float — WGS84
2	pickup_longitude	Float — WGS84
3	dropoff_latitude	Float — WGS84
4	dropoff_longitude	Float — WGS84
5	estimated_distance_km	From routing engine (OSRM / Google Maps)
6	estimated_duration_min	From routing engine
7	vehicle_category	economy / premium / xl / auto
8	toll_cost_estimate	From toll database for route; 0 if no toll

D. Contextual Modifier Features (Optional but Recommended)

#	Field	Description
1	rainfall_mm_per_hour	From weather API — rain increases demand
2	temperature_celsius	Ambient temperature for comfort-demand correlation
3	active_event_in_zone	Boolean — concert, match, festival near pickup
4	event_name	String label for logging (e.g. IPL Match, Concert)
5	city_id	Multi-city support — different base rates per city

5. Historical Data Requirements (For Model Training)

Please provide:

- Minimum 12 months of historical trip and zone-level data (18 months preferred)
- Minimum 1,000,000 completed trip records across all vehicle categories
- Data format: CSV export or direct PostgreSQL read access

Dataset must include:

- All fields listed in Section 4
- offered_surge_multiplier and final_fare_charged per trip
- rider_accepted_flag — did the rider proceed to book at the offered fare?
- booking_to_completion_flag — was the trip completed without cancellation?
- driver_id of assigned driver and time_to_assignment_sec
- zone_demand_at_request and zone_supply_at_request — snapshot at time of request
- actual_distance_km and actual_duration_min — post-trip GPS telemetry
- weather_condition at time of request (if available)
- active_event_flag and event_type for historical event-tagged trips

Data Quality Expectations

- No missing values in mandatory fields (pickup/dropoff coordinates, fare, acceptance flag)
- Surge multiplier values within valid range (1.0 – configured max cap)
- Consistent UTC timezone across all timestamps
- Driver and rider identifiers pseudonymized (hashed IDs)
- Estimated vs actual distance deviation flagged if > 150%
- Minimum 1,000,000 trip records required before model training begins

6. Kafka Integration Requirements

The Dynamic Pricing Engine will consume and produce data through Apache Kafka topics.

Required Kafka Topics (Consumed)

Please confirm availability and schema for:

- ride_requests — triggers per-request fare computation and zone demand increment
- driver_status — driver online/offline, location, availability updates (zone supply)
- trip_completed — for demand decrement and zone state recalibration post-trip
- event_calendar_updates — active city events near zones (surge context modifier)

Required Kafka Topics (Produced)

- surge_zone_updates — published every 60 seconds; consumed by Long Trip Model and Driver App
- fare_computed_events — audit log of every computed fare for analytics and dispute handling

Event Schema — ride_requests topic (pricing trigger)

```
{
  "request_id":              "string (UUID)",
  "timestamp":               "ISO8601",
  "rider_id":                "string (hashed)",
```

```
  "pickup_lat":               "float",
  "pickup_lon":               "float",
  "dropoff_lat":              "float",
  "dropoff_lon":              "float",
  "estimated_distance_km":    "float",
  "estimated_duration_min":   "integer",
  "vehicle_category":         "string (economy|premium|xl|auto)",
  "toll_cost_estimate":       "float"
}
```

Event Schema — surge_zone_updates topic (produced by engine)

```
{
  "zone_id":               "string (H3 hex ID)",
  "timestamp":            "ISO8601",
  "demand_supply_ratio": "float",
  "surge_multiplier":    "float",
  "surge_tier":           "string (normal|mild|moderate|peak|capped)",
  "active_drivers":       "integer",
  "active_requests":      "integer",
  "ttl_seconds":          "integer (60)"
}
```

Additional requirements — please confirm:

- Message serialization format (JSON / Avro / Protobuf)
- Kafka broker hostname, port, and authentication method
- Topic retention period and replication factor requirements

7. Model Input / Output Interface

Input Format (Fare Computation — from Booking Backend)

```
{
  "request_id":                     "string",
  "timestamp":                      "2026-03-24T18:00:00Z",
  "pickup_lat":                     19.0760,
  "pickup_lon":                     72.8777,
  "dropoff_lat":                    19.1136,
  "dropoff_lon":                    72.8697,
  "estimated_distance_km":          12.4,
  "estimated_duration_min":         28,
  "vehicle_category":               "economy",
  "toll_cost_estimate":             0.0,
  "hour_of_day":                    18,
  "is_peak_hour":                   true,
  "is_holiday":                     false,
  "zone_demand_supply_ratio":       2.1,
  "rainfall_mm_per_hour":           8.5,
  "active_event_in_zone":           false,
  "available_drivers_in_zone":      4
}
```

Output Format (from Dynamic Pricing Engine)

```
{
  "request_id":           "string",
  "surge_multiplier":     1.6,
  "surge_tier":           "moderate",
  "final_fare":           187.0,
  "fare_breakdown": {
    "base_fare":          112.0,
    "surge_component":    67.2,
    "toll_component":     0.0,
    "platform_fee":       7.8
  },
  "currency":             "INR",
  "fare_valid_until":     "2026-03-24T18:02:00Z",
  "surge_banner_text":    "High demand in your area",
  "is_surge_capped":      false
}
```

8. Integration Requirements

Please confirm:

- How the backend will call the pricing engine:
  - REST API (preferred) — synchronous call per ride_request event from Booking Service
  - Internal gRPC call as alternative for lower latency
- Expected response time:
  - Target: < 100 ms per fare computation (P99 SLA)
  - Zone surge state batch refresh: every 60 seconds
- Deployment approach:
  - Separate microservice — decoupled from Booking Service (recommended)
  - Horizontally scalable; stateless inference layer with zone state in Redis
- Downstream integrations:
  - Booking Service — consumes final_fare and fare_breakdown at ride request
  - Driver App — consumes surge_zone_updates Kafka topic for surge indicator display
  - Long Trip Optimization Model — consumes zone_surge_multiplier as input feature
  - Analytics Warehouse — receives fare_computed_events for BI dashboards

9. Technology Alignment

The Dynamic Pricing Engine will be built using the following technology stack. Please confirm compatibility with the existing Urban Black backend architecture.

Component	Technology / Tool
Routing Engine	OSRM (self-hosted) or Google Maps Distance Matrix API
ML Framework	LightGBM (surge multiplier regression model)
Geospatial Zoning	H3 (Uber’s hexagonal hierarchical geospatial index)
Inference API	FastAPI (Python) — REST endpoint for fare computation
Zone State Cache	Redis — in-memory zone demand-supply snapshots
Event Streaming	Apache Kafka — ride_requests, driver_status, surge_zone_updates
Database	PostgreSQL — historical trip data, base fare config, toll master

Containerisation	Docker + Kubernetes — scalable microservice deployment
Experiment Tracking	MLflow — model versioning and experiment registry
Monitoring	Prometheus + Grafana — surge multiplier drift and API latency
Holiday/Event Calendar	Internal city calendar service or Google Calendar API

10. Evaluation Criteria

The Dynamic Pricing Engine will be evaluated against the following performance benchmarks. Please confirm acceptable thresholds with the business and product team.

Metric	Target
Fare MAE (vs actual accepted fare)	< ₹15 per trip
Rider fare acceptance rate	> 80%
Surge multiplier prediction RMSE	< 0.15
API response time (P99)	< 100 ms
Zone state refresh latency	< 60 seconds
Driver assignment time (indirect)	< 90 seconds post fare display
Booking conversion rate (surge vs normal)	Decline < 15% during mild surge
Revenue per trip lift vs flat-rate baseline	> 10%
System uptime SLA	99.9% (three nines)

11. Deployment Requirements

- Containerised deployment using Docker + Kubernetes
- Inference triggered synchronously per ride request via REST API from Booking Service
- Zone surge state batch refresh every 60 seconds via scheduled Kafka consumer
- Redis cache populated by zone refresh job; read by inference API for low-latency response
- Model retraining schedule: bi-weekly (surge model) + monthly (full pipeline)
- Environment details required: dev / staging / prod Kafka cluster and database endpoints
- Cloud or on-prem infrastructure access — please confirm provider
- Surge cap configuration must be externalised (not hardcoded) — configurable per city via admin panel
- Regulatory override mechanism required: ability to disable surge for government-mandated events

12. Dependencies Required from Team

Please provide:

- Historical trip dataset — minimum 12 months (CSV or PostgreSQL read access)
- Base fare matrix per vehicle category and city (distance and time components)
- Toll cost database per corridor / road segment
- City zone map (H3 hex grid or equivalent geofencing polygons)
- OSRM instance or Google Maps API credentials for distance/ETA calculation
- Kafka topic details: broker address, topic names, schemas, and auth credentials
- Driver status event schema and driver availability feed
- City holiday and public event calendar (API or flat file)

- Weather API access (OpenWeatherMap or equivalent) for rainfall signal
- Business pricing rules: base fare formula, platform fee %, surge cap per city, minimum fare
- Backend API contract: confirmation of input/output JSON schema per Section 7

13. Assumptions

- Surge cap is set at 2.5× by default; configurable per city without code deployment
- Base fare formula (per km + per minute + fixed base) is provided by the business team and stored in PostgreSQL config table
- Zone grid resolution (H3 level 7) may be adjusted per city density — to be confirmed during onboarding
- Driver location updates arrive via driver_status Kafka topic at minimum every 30 seconds
- Weather data is fetched at zone level every 5 minutes; stale data older than 15 minutes uses last known value
- Toll database is refreshed monthly; manual admin override available for emergency route changes
- The fare quote is valid for 2 minutes after display; expired quotes trigger a re-computation
- Rider-facing surge banner text is managed by the frontend team; the engine returns a raw text string
- Multi-city support is required from Day 1 — all config (base fare, caps, zones) is parameterised by city_id
- Minimum 1,000,000 historical trip records required before supervised ML model training begins; rule-based fallback will be used during cold start

14. Summary

This document outlines the complete requirements needed to build a production-ready Dynamic Pricing Engine for Urban Black. The engine will compute real-time zone-level surge multipliers and rider-facing fare estimates at booking time, factoring in demand-supply dynamics, time-of-day patterns, weather conditions, and active city events.

The system integrates with the Booking Service via synchronous REST API for per-request fare computation, and publishes zone surge state to Kafka for downstream consumption by the Long Trip Optimization Model, Driver App, and analytics warehouse. A Redis-backed zone state cache ensures sub-100 ms response times under peak load.

The engine is designed as a horizontally scalable, containerised microservice with configurable surge caps, regulatory override capability, and a bi-weekly ML retraining pipeline. The ML model will be trained on historical trip data once the 1,000,000 trip threshold is reached; a rule-based computation engine will handle fare calculation during the cold-start period.